

YadYar Lite - Phase 1 Report

T-05: Retrieval-Augmented Generation and the Study of Hallucination

Team members: Taha Amini, Eiliya Yavari

Course: Artificial Intelligence and Expert systems, Spring 1404-1405

Instructor: Dr. Koohzadi

1. Project Question

How well can a lightweight retrieval-augmented generation (RAG) system find answer evidence in a small document collection and avoid unsupported answers when the available text does not contain an answer?

The project studies the two main parts of a RAG pipeline separately. First, we measure whether the retriever finds a chunk that contains the correct answer. The retriever is now connected to a local generator, and Phase 2 will measure whether the generator uses the retrieved evidence correctly and returns UNANSWERABLE when the evidence is not enough. This separation helps us understand whether an incorrect final answer was caused by retrieval or generation.

2. Dataset Description

We use a reproducible subset of the SQuAD 2.0 development set. SQuAD 2.0 is a question-answering dataset based on Wikipedia passages. It contains both answerable questions and questions whose answers do not exist in the related passage. This makes it suitable for studying retrieval as well as hallucination and abstention.

The prepared subset has the following size:

Item	Value
Source contexts	56
Total words in the contexts	7,144
Retrieval chunks	146
Chunk size	70 words
Chunk overlap	15 words
Answerable questions	50
Unanswerable questions	50

Each context has an ID and its original text. Each chunk contains a chunk ID, the ID of its source context, and the chunk text. Each evaluation question contains its SQuAD ID, question text, source context ID, reference answers, and an `is_answerable` label.

SQuAD 2.0 was selected for three reasons. First, it is a public and commonly used question-answering dataset. Second, it provides reference answers that can be used for automatic evaluation. Third, its unanswerable questions allow us to test whether the system invents an answer when evidence is missing.

3. Data Preparation

The data is prepared by `rag_engine/data/prepare_data.py`. The script performs these steps:

1. It downloads the official SQuAD 2.0 development file if the file is not already available.
2. It extracts the Wikipedia paragraphs and their questions.
3. It shuffles the paragraphs with random seed 42.
4. It selects paragraphs until the sample contains at least 7,000 words.
5. It splits the selected contexts into 70-word chunks with a 15-word overlap.
6. It selects 50 answerable and 50 unanswerable questions.
7. It saves the processed data in three JSON files.

The generated files are:

```
rag_engine/data/sample_contexts.json
rag_engine/data/sample_chunks.json
rag_engine/data/sample_questions.json
```

Using a fixed seed makes the selection repeatable. The overlap is used to reduce the chance that important evidence is lost at a chunk boundary. The raw dev-v2.0.json file is not committed because it can be downloaded again by the script.

4. Baseline Setup

The implemented Phase 1 baseline is the retrieval component of the RAG system. It uses BAAI/bge-base-en-v1.5, an existing pretrained embedding model, to convert chunks and questions into vectors. Chunk vectors are stored in ChromaDB. The database uses cosine distance, and the three closest chunks are returned for every question (Top-k = 3).

The complete lightweight pipeline is:

Question

- > question embedding
- > ChromaDB cosine search
- > Top-3 chunks
- > generator
- > answer or UNANSWERABLE

The generator uses the pretrained qwen2.5:3b model through a local Ollama service. It receives the question and the three retrieved chunks, and its prompt instructs it to answer only from those chunks or return exactly UNANSWERABLE when they do not provide enough evidence. The end-to-end pipeline is available through main.py. No model is trained from scratch and no fine-tuning is used.

The retrieval baseline can be reproduced from the repository root with:

```
python -m pip install -r requirements.txt
python rag_engine/data/prepare_data.py
python rag_engine/evaluation/evaluate_retrieval.py
```

To run the current end-to-end demo, Ollama must be running locally and the generator model must be available:

```
ollama pull qwen2.5:3b
python main.py
```

The evaluator clears and rebuilds a separate ChromaDB collection named rag_evaluation on every run. This prevents old or duplicate data from changing the result. Detailed results are saved in rag_engine/evaluation/retrieval_results.json.

As an initial reproducibility check, the evaluator was run on the 50 answerable questions. Evidence Recall@3 was 0.98, so answer evidence was found for 49 of 50 questions. This is an initial baseline check; the complete end-to-end results will be reported in Phase 2 after generation evaluation is completed.

5. Evaluation Plan

We will use two main metrics.

5.1 Evidence Recall@3

Evidence Recall@3 evaluates the retriever on answerable questions. A question is counted as successful when at least one normalized reference answer appears inside one of the three retrieved chunks.

$$\text{Evidence Recall@3} = \frac{\text{questions with answer evidence in the Top-3 chunks}}{\text{all answerable questions}}$$

This is stricter than checking only the source context ID. A retrieved chunk may come from the correct paragraph but still not contain the sentence with the answer. Context Recall and Recall@1 are also stored as diagnostic values, while Evidence Recall@3 is the main retrieval metric.

5.2 Answer F1

The connected generator’s final answers will be compared with the SQuAD reference answers using token-level F1. Before comparison, answers will be converted to lowercase and punctuation, English articles, and extra spaces will be removed. F1 gives partial credit when the generated answer and a reference answer overlap but are not exactly the same.

The final evaluation will include an answerable versus unanswerable breakdown. For the 50 unanswerable questions, returning UNANSWERABLE is considered a correct abstention. We will also calculate hallucination rate as a simple diagnostic:

$$\text{Hallucination Rate} = \frac{\text{unanswerable questions given a normal answer}}{\text{all unanswerable questions}}$$

6. Simple Analysis Plan

We will manually inspect representative failures and group them into three simple categories:

1. **Retrieval failure:** none of the Top-3 chunks contains the answer evidence. Possible causes include paraphrase mismatch, an ambiguous question, or a chunk boundary.
2. **Ignored-evidence hallucination:** the correct evidence is retrieved, but the generator produces an incorrect or unsupported answer.
3. **Absent-evidence hallucination:** the answer is not supported by the retrieved text, but the generator answers instead of returning UNANSWERABLE.

We will also compare Evidence Recall@3 for short and long questions. The median question length is used as the dividing point. In the initial run, the median was 9 words: the short-question group obtained 28/29 successful retrievals, while the long-question group obtained 21/21. The single failed question used the ambiguous phrase “this settlement” without naming the settlement, which made semantic retrieval difficult.

7. Limitations

- The corpus and evaluation sample are small, so the results cannot represent all open-domain question-answering tasks.
- Fixed word-based chunking may split a sentence or separate an answer from useful surrounding information.
- Answer-in-chunk matching may miss evidence that expresses the same meaning using different words.
- Some unanswerable questions could accidentally be answered from another passage in a larger corpus.
- The final results depend on the embedding model, generator, prompt, chunking settings, and the selected value of Top-k.

8. AI Tool Usage

OpenAI ChatGPT was used to help organize the preprocessing script, write the retrieval evaluation code, and structure this report. The team reviewed the code, ran the evaluation locally, checked the generated files and results, and is responsible for explaining the implementation and design decisions. The dataset and reference answers come from SQuAD 2.0, not from ChatGPT.

References

1. Rajpurkar, P., Jia, R., and Liang, P. (2018). Know What You Don’t Know: Unanswerable Questions for SQuAD. ACL.
2. SQuAD Explorer: <https://rajpurkar.github.io/SQuAD-explorer/>
3. BGE model page: <https://huggingface.co/BAAI/bge-base-en-v1.5>
4. ChromaDB documentation: <https://docs.trychroma.com/>